

Pathtracer report

This report aims at presenting the results obtained while working on the raytracer project. We present our results, along with the generated images, rendering times and explanations on why the code works. The renderer handles ray-sphere intersection, direct and indirect illumination, anti-aliasing, depth of field, mirror reflections, triangle meshes and BVH acceleration. All renders use 512×512 pixels with gamma correction $\gamma = 2.2$

Ray-sphere intersection and direct lighting

Rays are emitted from a pinhole camera looking down $-z$. For each pixel (j, i) , the ray direction is

$$\text{normalize} \begin{pmatrix} j - \frac{W}{2} + 0.5 \\ \frac{H}{2} - i - 0.5 \\ -\frac{W}{2 \tan(\frac{\alpha}{2})} \end{pmatrix}$$

A ray $O + tu$ hits a sphere of center C and radius R when the discriminant is non-negative

$$\Delta = (u \cdot (O - C))^2 - \|O - C\|^2 + R^2$$

Then the two candidate distances are given by $u \cdot (C - O) \pm \sqrt{\Delta}$. Both roots are tested in order, and we take the smallest non-negative one, which gives us the hit point $P = O + tu$ and normal $N = \frac{P - C}{\|P - C\|}$. The scene loop keeps the closest hit across all objects.

The direct light follows the Lambertian model:

$$\begin{aligned} L_{\text{direct}} &= \frac{I}{4\pi d^2} \frac{\rho}{\pi} V_P(L) \langle N, \omega_i \rangle \\ &= \frac{I}{4\pi \|L - P\|^2} \cdot \frac{\rho}{\pi} \cdot \max\left(0, N \cdot \frac{L - P}{\|L - P\|}\right) \end{aligned}$$

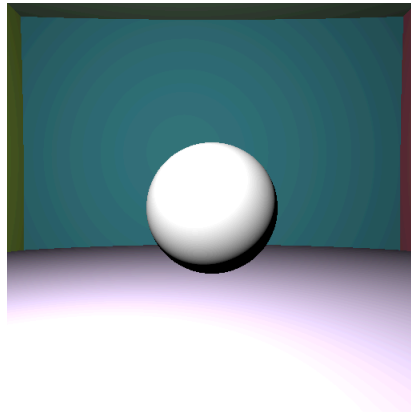


Figure 1: The sphere with direct lighting only (0.142s).

Shadows

For shadows, a ray is shot from $P + \varepsilon N$ towards L . If it hits any object before the light, then the point is in the shadow.

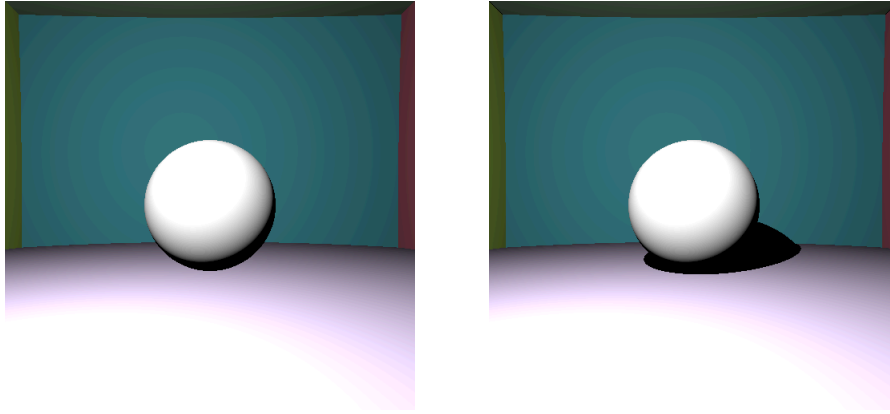


Figure 2: Before (0.142s) and after adding shadows (0.203s).

Mirror reflections

When a ray hits a mirror surface, the reflected direction is

$$u - 2(u \cdot N)N$$

Then we recursively trace a new ray from $P + \varepsilon N$ in this direction.

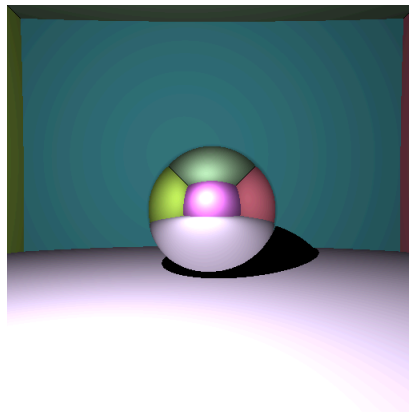


Figure 3: Enabling mirror reflections on the sphere (0.227s).

Indirect lighting

The rendering equation for a diffuse surface is

$$L_{\text{direct}} + \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \langle \omega_i, N \rangle d\omega_i$$

We estimate the integral with one cosine-weighted Monte Carlo sample per bounce. As explained in the lecture notes, when performing integration, the cosine terms cancel out, as well as the factor π . Rays are stopped after 5 bounces to keep rendering times reasonable, and because each path uses only one random direction per bounce, we average 32 samples per pixel to reduce noise.

To implement the `random_cos` function, we build the tangent frame (T_1, T_2, N) by first choosing T_1 along the axis where N is the smallest, then set $T_2 = N \times T_1$, and finally draw two uniform random numbers r_1 and r_2 .

We have

$$\omega_i = \sqrt{1 - r_2} \cos(2\pi r_1) T_1 + \sqrt{1 - r_2} \sin(2\pi r_1) T_2 + \sqrt{r_2} N$$

Random numbers are sampled using per-thread generators to avoid data races with OpenMP.

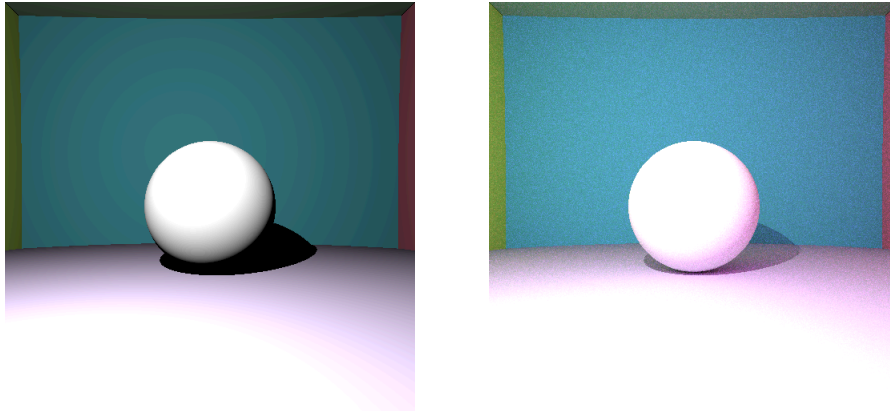


Figure 4: Before (0.203s) and after adding indirect lighting (9.926s).

Antialiasing

As we are always sampling rays in the middle of each pixel, the abrupt variations between adjacent pixels produces aliasing. To eliminate these artifacts, we add a per-sample Gaussian offset $\sigma = 0.5$ using the Box-Muller transform

$$x = \sqrt{-2 \ln(r_1)} \cos(2\pi r_2) \sigma, \quad y = \sqrt{-2 \ln(r_1)} \sin(2\pi r_2) \sigma \quad \text{where } r_1, r_2 \sim \mathcal{U}(0, 1)$$

By averaging 32 perturbed samples per pixel, we essentially apply a Gaussian filter.

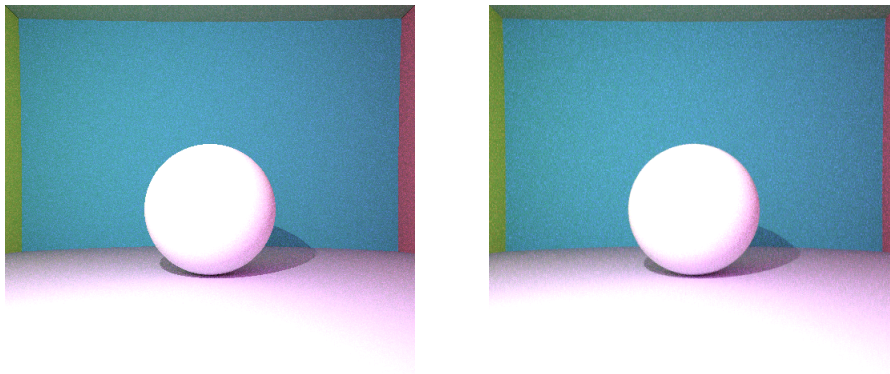


Figure 5: Before (9.926s) and after adding antialiasing (10.149s).

Depth of field

A pinhole camera renders everything perfectly sharp. We add a circular aperture of radius 0.5 to simulate the blurring that occurs with a real camera. Instead of originating from a single point, each ray now starts at a randomly sampled position on the aperture disc, and is directed towards the focal point, where the original pinhole ray would intersect the focal plane at distance d . Then points exactly at distance d stay sharp, while others are blurred proportionally to how far they are from the focal plane.

We pick points uniformly on the aperture by setting

$$r = 0.5\sqrt{r_1}, \quad \theta = 2\pi r_2 \quad \text{where } r_1, r_2 \sim \mathcal{U}(0, 1)$$

The square root prevents clustering near the center. Finally, we set the focus distance to 55, to match the distance between the camera and the scene.

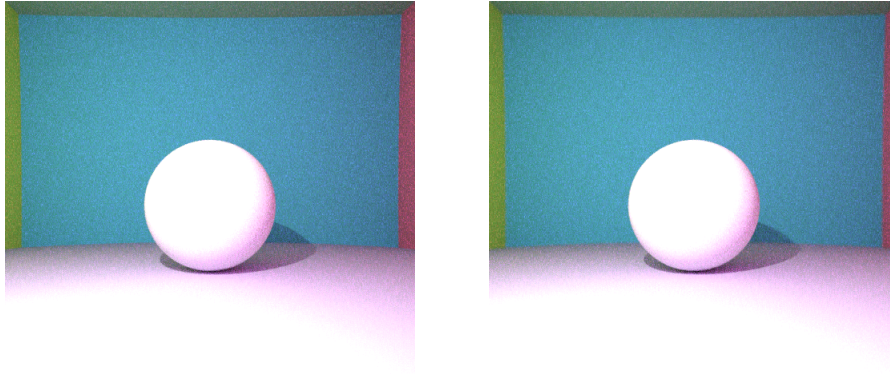


Figure 6: Before (10.149s) and after adding depth of field (10.480s).

Triangle meshes

Before testing individual triangles, we first intersect the ray with the bounding box of the object. The lecture notes state that the ray hits the box if and only if

$$\min(t_1^x, t_1^y, t_1^z) > \max(t_0^x, t_0^y, t_0^z)$$

If the condition evaluates to false, the ray misses the bounding box, and because this box encapsulates the object entirely, it guarantees that no triangle can be intersected, so we can safely skip the computations.

We compute the bounding box once after translating and scaling the object, and store it for later use (recomputing it each time was an initial mistake, and would negate the benefits almost entirely, or even completely in some cases).

We compute triangle intersection using the Muller-Trumbore algorithm. For a triangle (A, B, C) with edges $e_1 = B - A$ and $e_2 = C - A$ we have

$$t = \frac{(A - O) \cdot (e_1 \times e_2)}{u \cdot (e_1 \times e_2)}$$

$$\beta = \frac{e_2 \cdot ((A - O) \times u)}{u \cdot (e_1 \times e_2)}, \quad \gamma = -\frac{e_1 \cdot ((A - O) \times u)}{u \cdot (e_1 \times e_2)}, \quad \alpha = 1 - \beta - \gamma$$

The hit is only valid when $t, \alpha, \beta, \gamma \geq 0$. The normal is then computed as

$$N = \frac{e_1 \times e_2}{\|e_1 \times e_2\|}$$

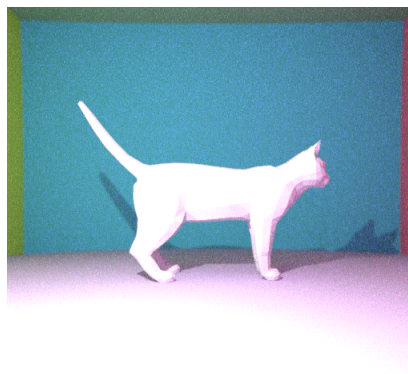


Figure 7: The cat, with the bounding box optimization enabled (266.412s).

BVH acceleration

For each one of the 512×512 pixels in the image, we need to compute 32 intersections for each of the 3954 faces of the cat object. We implement a Bounding Volume Hierarchy to reduce the complexity of computing the intersection from $O(n)$ to $O(\log n)$.

Each node stores a bounding box and a triangle range. The construction is top-down: we compute the bounding box of the current range, split along the longest axis at its midpoint, and partition triangles by their centroid. Nodes with less than or exactly 4 triangles become leaves. If all centroids fall on the same side, we fall back to splitting at the index midpoint so recursion always terminates. The longest-axis split logic was discussed with Léonard Mauvernay.

We run the test on each node recursively. Whenever the condition evaluates to false, we discard the whole subtree. Whenever the condition evaluates to true, inner nodes recurse into both children, and leaves test all their triangles with Muller-Trumbore.

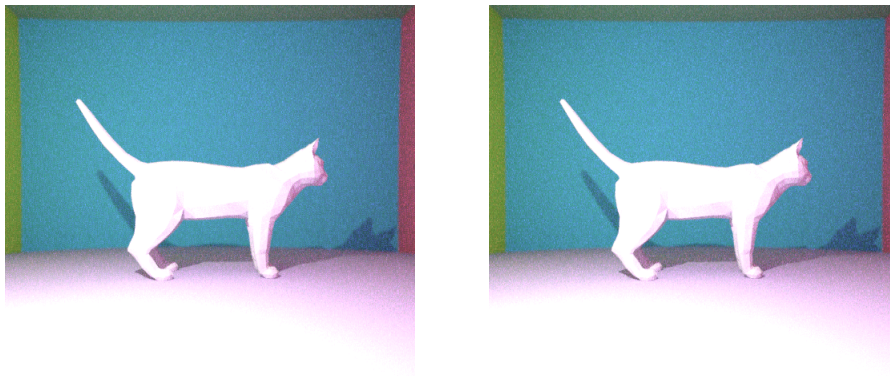


Figure 8: Before (266.412s) and after enabling the BVH optimization (11.744s).

Timings

The table below summarizes render times as each feature is added on top of the previous one. All timings are for a 512×512 render, with 32 samples per pixel and 5 bounces. For each feature, we render 10 times and report the average.

Feature	Without	With
Direct lighting	—	0.142s
Shadows	0.142s	0.203s
Mirror reflections	0.203s	0.227s
Indirect lighting	0.203s	9.926s
Antialiasing	9.926s	10.149s
Depth of field	10.149s	10.480s
Bounding box	1757.488s	266.412s
Bounding Volume Hierarchy	266.412s	11.744s